SQLD 제59회 기출 복원문제

각 문항 아래에 정답과 선택지별 풀이를 바로 확인할 수 있습니다.

정답·상세 풀이
총 50문항 · 1과목 10 / 2과목 40

1과목 데이터 모델링의 이해

01 1과목

데이터 모델링의 특징에 해당하지 않는 것은?

① 추상화(Abstraction)
② 단순화(Simplification)
③ 명확화(Clarity)
④ 최적화(Optimization)

정답 ④

핵심 해설

데이터 모델링의 세 가지 특징은 추상화·단순화·명확화이다.

선택지별 풀이

① 추상화는 현실 세계를 일정한 형식(표기법)으로 간략히 표현하는 것,
② 단순화는 누구나 이해할 수 있도록 약속된 규약과 표기로 간결하게 표현하는 것,
③ 명확화는 애매모호함을 제거하고 정확하게 기술하는 것이다.
④ 최적화는 성능·자원 관점의 튜닝 개념으로 모델링의 본질적 특징이 아니므로 정답이다. 즉 모델링은 '무엇을' 정확히 표현하느냐(추상화·단순화·명확화)에 관한 것이지 성능 최적화 자체를 특징으로 삼지 않는다.

02 1과목

어느 은행이 예금 상품을 관리하기 위해 예금분류 엔터티를 두고, 각 예금의 원금·예치기간·이자율·계산이자를 관리하며 예금 종류마다 코드를 부여한다. 이때 속성 분류에 대한 설명으로 옳지 않은 것은?

① 예금분류 코드는 별도의 코드 엔터티로 분리해 관리하고, 예금 엔터티에는 코드 값만 저장한다.
② 원금과 예치기간은 업무로부터 직접 입력받는 기본속성이다.
③ 이자율과 예치기간은 다른 속성으로부터 계산되어 나오는 파생속성이다.
④ 계산이자는 원금·이자율·예치기간으로부터 산출되는 파생속성이다.

정답 ③

핵심 해설

속성은 성질에 따라 기본속성(업무로부터 직접 발생·입력), 설계속성(설계 과정에서 새로 만든 코드성 속성), 파생속성(다른 속성을 계산·가공하여 만든 속성)으로 나뉜다.

선택지별 풀이

④ 계산이자는 원금×이자율×기간으로 산출되므로 파생속성이 맞다.
② 원금·예치기간은 계약 시 직접 입력되는 기본속성이 맞다.
① 코드성 값을 별도 코드 엔터티로 분리하고 참조 값만 저장하는 것은 설계속성/코드 관리의 전형이다.
③ 이자율은 상품 조건으로 주어지는 기본속성이고 예치기간도 기본속성이므로, 둘을 파생속성이라 한 ③이 틀렸다. 파생속성은 '계산 결과'인 계산이자에 해당한다.

03 1과목

각 속성이 가질 수 있는 값의 범위, 즉 데이터 타입·크기·허용 값의 집합을 정의한 것을 무엇이라 하는가?

① 제약조건
② 도메인
③ 식별자
④ 관계

정답 ②

핵심 해설

② 도메인(Domain)은 각 속성이 가질 수 있는 값의 범위를 의미하며, 데이터 타입·길이·허용 값 목록 등으로 정의된다. 예컨대 '성별' 속성의 도메인은 {남, 여}, '점수'의 도메인은 0~100의 정수 등이다.

선택지별 풀이

① 제약조건은 데이터 무결성을 지키기 위한 규칙(NOT NULL, UNIQUE 등)으로 도메인보다 넓은 개념이다.

③ 식별자는 엔터티의 인스턴스를 유일하게 구분하는 속성(집합)이다.

④ 관계는 엔터티 간의 연관성을 의미한다. 따라서 값의 범위 정의는 도메인이다.

04 1과목

ERD에서 관계(Relationship)를 표현할 때 사용하는 표기 요소가 아닌 것은?

① 관계명(Relationship Membership)

② 관계차수(Cardinality)

③ 관계선택사양(Optionality)

④ 관계빈도(Relationship Frequency)

정답 ④

핵심 해설

관계를 표기할 때 다루는 요소는 ① 관계명(두 엔터티가 어떤 관계인지 명명),

선택지별 풀이

② 관계차수(1:1, 1:M, M:N 등 카디널리티),

③ 관계선택사양(필수/선택, Optional·Mandatory)이다.

④ 관계빈도는 표준 관계 표기 요소가 아니므로 정답이다. 관계 표기 3요소는 '관계명·차수·선택사양'으로 암기하면 된다. 빈도는 트랜잭션 발생 횟수 등 성능 분석 관점의 용어일 뿐 관계의 구조적 표기 요소가 아니다.

05 1과목

다음 중 엔터티(Entity)의 특징으로 옳지 않은 것은?

① 엔터티는 반드시 하나의 인스턴스만 가진다.

② 엔터티는 두 개 이상의 속성을 가진다.

③ 각 인스턴스를 유일하게 구분하는 식별자를 가진다.

④ 업무에서 사용되며 다른 엔터티와 관계를 가진다.

정답 ①

핵심 해설

정답(옳지 않은 것)은 ①이다. 엔터티는 두 개 이상의 인스턴스의 집합이어야 하므로 "하나의 인스턴스만 가진다"는 틀린 설명이다.

선택지별 풀이

② 엔터티는 두 개 이상의 속성을 가진다(옳음).

③ 각 인스턴스를 유일하게 식별하는 식별자를 가진다(옳음).

④ 업무에서 사용되고 다른 엔터티와 관계를 맺는다(옳음).

> **보충 개념** 엔터티의 특징 — 유일한 식별자, 2개 이상 인스턴스, 2개 이상 속성, 다른 엔터티와의 관계, 업무에서의 사용.

06 1과목

A-B-C-D 네 엔터티가 각각 1:M 식별자 관계로 연쇄 연결된 모델이 있다. 식별관계와 비식별관계에 대한 설명으로 옳지 않은 것은?

① 식별관계에서는 부모 엔터티의 기본키가 자식 엔터티의 기본키의 일부(구성 속성)로 상속된다.

② 비식별관계로 변경하면 상위 엔터티와의 조인 없이도 데이터에 접근할 수 있어 조회 성능이 항상 향상된다.

③ 비식별관계에서는 부모 엔터티의 기본키가 자식 엔터티의 일반 속성(외래키)으로 상속된다.

④ 식별관계가 연쇄되면 하위 엔터티일수록 기본키를 구성하는 속성의 개수가 늘어난다.

정답 ②

핵심 해설

식별관계는 부모 PK가 자식 PK의 일부로 상속되어 PK가 대물림되며(①·④ 옳음), 이 때문에 하위 엔터티에서 상위 키 값을 조인 없이 바로 참조할 수 있어 조회에 유리한 경우가 많다. 반대로 비식별관계는 부모 PK가 자식의 '일반 속성(FK)'으로만 내려오므로(③ 옳음), 상위 데이터를 얻으려면 오히려 조인이 더 필요해질 수 있다. 따라서 '비식별로 바꾸면 조인 없이 접근되어 조회 성능이 항상 향상된다'는 ②는 인과가 뒤바뀐 서술로 옳지 않다(정답). 비식별관계는 PK 전파를 끊어 모델을 단순화하지만, 상위 정보 조회 시 조인이 늘 수 있다는 트레이드오프가 있다.

07 1과목

다음 [주문] 테이블에서 사원번호 → 사원명, 사원부서 함수 종속이 존재한다. 기본키(주문번호) → 사원번호 → 사원명·사원부서 형태의 이행적 함수 종속을 제거하기 위해 수행해야 하는 정규화 단계는?

주문번호(PK)	주문일자	사원번호	사원명	사원부서
1001	2026-01-02	E01	홍길동	영업1팀
1002	2026-01-03	E01	홍길동	영업1팀

① 제1정규화 ② 제2정규화 **③ 제3정규화** ④ 반정규화

정답 ③

핵심 해설

이행적 함수 종속(A→B, B→C이므로 A→C)을 제거하는 단계는 제3정규화이다(정답

선택지별 풀이

③). 여기서는 주문번호→사원번호, 사원번호→사원명·사원부서로 이어지는 이행 종속이 있으므로, [사원] 엔터티(사원번호 PK, 사원명, 사원부서)를 분리하고 [주문]에는 사원번호만 FK로 남긴다.

① 제1정규화는 반복·다중값 속성을 제거해 원자값으로 만드는 단계,

② 제2정규화는 복합키에 대한 부분 함수 종속을 제거하는 단계이다.

④ 반정규화는 성능을 위해 정규화를 일부 되돌리는 것으로 종속 제거와 무관하고, 개정 출제범위에서 상세 기법은 SQLP로 이관되었다. 따라서 이행 종속 제거는 제3정규화이다.

08 1과목

트랜잭션(Transaction)이 가져야 할 특성(ACID)에 해당하지 않는 것은?

① 원자성(Atomicity) **② 보안성(Security)**

③ 일관성(Consistency) ④ 영속성(Durability)

정답 ②

핵심 해설

트랜잭션의 4대 특성 ACID는 원자성(Atomicity, 모두 반영되거나 모두 취소), 일관성(Consistency, 실행 전후 데이터 무결성 유지), 고립성(Isolation, 수행 중 다른 트랜잭션이 끼어들 수 없음 — 교재에 따라 격리성·독립성으로도 번역), 영속성(Durability, 완료된 결과는 영구 보존)이다.

②의 '보안성(Security)'은 데이터베이스 보안 영역의 개념일 뿐 ACID의 특성이 아니므로 정답이다. 나머지 ①·③·④는 모두 ACID의 정식 항목이다.

09 1과목

식별자를 그 대표성·생성 방식에 따라 본질식별자와 인조식별자로 구분할 때, 설명으로 옳지 않은 것은?

① 인조식별자는 업무로부터 자연스럽게 만들어진 식별자로서 그 자체가 업무 규칙과 의미를 표현한다.

② 본질식별자는 업무에 의해 자연스럽게 만들어지는 식별자이다.

③ 주식별자가 지나치게 많은 속성으로 구성될 때 이를 대체하기 위해 인조식별자를 사용할 수 있다.

④ 인조식별자는 일련번호처럼 인위적으로 부여한 식별자이다.

정답 ①

핵심 해설

본질식별자는 업무에 의해 자연스럽게 만들어지는 식별자(②·맞음)이고, 인조식별자는 본질식별자가 복잡하거나 속성이 많을 때(③·맞음) 이를 대체하려고 일련번호 등으로 인위적으로 부여한 식별자(④·맞음)이다. 인조식별자는 업무적 의미가 없고 단지 유일성 확보용이므로, '업무로부터 자연스럽게 만들어져 업무 규칙과 의미를 표현한다'고 서술한 ①은 인조식별자가 아니라 본질식별자의 특징을 잘못 붙인 것으로 옳지 않다(정답). 인조식별자 도입 시 불필요한 인덱스가 늘 수 있고 본질식별자의 업무 규칙이 숨겨지는 단점이 있다.

10 1과목

다음 ERD의 [학생]—[수강] 관계를 읽은 것이다. 관계선은 학생 쪽 끝이 '실선(필수)'이고 수강 쪽 끝이 '까마귀발+실선(1개 이상 필수)'으로 표기되어 있다. 이 관계에 대한 해석으로 가장 옳은 것은? [학생] ||————————< [수강]

① 한 명의 학생은 수강 신청을 하지 않을 수도 있다.

② 하나의 수강 인스턴스는 여러 학생과 연결될 수 있다.

③ 한 명의 학생은 최대 한 개의 수강만 가질 수 있다.

④ 한 명의 학생은 반드시 최소 한 개 이상의 수강을 해야 한다.

정답 ④

핵심 해설

까마귀발(Crow's Foot) 표기에서 학생 쪽이 필수(실선)이고 수강 쪽이 '까마귀발+필수' 표기이면, 학생 : 수강 = 1 : M이며 학생은 최소 1개 이상의 수강을 반드시 가져야 한다는 필수 참여를 의미한다. 따라서

선택지별 풀이

④ 가 옳다.

① '하지 않을 수도 있다'는 선택 참여(원, Optional) 표기일 때의 해석이므로 필수 표기와 배치되어 틀렸다.

② 하나의 수강이 여러 학생과 연결된다는 것은 M:N 또는 방향이 뒤바뀐 해석으로, 이 1:M 모델에서는 하나의 수강이 한 학생에 속한다.

③ '최대 한 개'는 1:1 해석으로 까마귀발(다수) 표기와 모순된다. 관계선은 '차수(1:M)'와 '선택사양(필수)'을 함께 읽어야 한다.

2과목 SQL 기본 및 활용

11 2과목

다음 DCL(데이터 제어어)에 대한 설명 중 옳은 것으로만 짝지어진 것은?

가. DCL은 트랜잭션을 제어하는 명령어로, COMMIT과 ROLLBACK이 이에 속한다.

나. GRANT 문으로 다른 사용자에게 권한을 부여할 수 있으며, WITH GRANT OPTION을 지정하면 부여받은 권한을 다시 다른 사용자에게 부여할 수 있다.

다. REVOKE는 이미 부여한 권한을 회수하는 명령어로, 실행되면 변경된 데이터를 물리 디스크에 영구 저장한다.

라. ROLE은 여러 개의 권한을 하나로 묶어 관리하는 권한의 집합으로, 사용자에게 ROLE을 부여하면 묶인 권한을 한 번에 부여할 수 있다.

① 가, 나 ② 나, 다 **③ 나, 라** ④ 가, 나, 라

정답 ③

핵심 해설

정답은 ③ 나, 라이다. 나: GRANT는 권한을 부여하는 DCL이며, WITH GRANT OPTION을 붙이면 권한을 받은 사용자가 그 권한을 제3자에게 재부여(위임)할 수 있다. 옳은 설명이다. 라: ROLE은 여러 시스템/객체 권한을 하나로 묶은 권한의 집합(권한 그

룹)으로, 사용자·ROLE 관리를 단순화한다. 옳은 설명이다. 가: COMMIT·ROLLBACK은 트랜잭션 제어어(TCL)이지 DCL이 아니다. DCL은 GRANT·REVOKE이다. 다: REVOKE는 권한 회수 명령일 뿐, 데이터를 물리 디스크에 저장하는 것과는 무관하다. 데이터의 영구 저장(반영)은 COMMIT(TCL)의 역할이다.

> **보충 개념** SQL 명령어는 DDL(CREATE/ALTER/DROP/TRUNCATE), DML(SELECT/INSERT/UPDATE/DELETE), DCL(GRANT/REVOKE), TCL(COMMIT/ROLLBACK/SAVEPOINT)로 분류된다.

12 2과목

사원들을 급여 오름차순으로 정렬했을 때, 각 사원의 바로 직전(앞) 행의 급여를 함께 출력하려고 한다. (㉠)에 들어갈 윈도우 함수로 가장 적절한 것은?

```
SELECT 사원명, 급여,
(㉠) AS 직전급여
FROM 사원;
```

① LAG(급여) OVER (ORDER BY 급여)

② LAG(급여) OVER (ORDER BY 급여 DESC)

③ LEAD(급여) OVER (ORDER BY 급여)

④ LEAD(급여) OVER (ORDER BY 급여 DESC)

정답 ①

핵심 해설

정답은 ①이다. LAG(컬럼)는 현재 행을 기준으로 정렬 순서상 '이전(앞) 행'의 값을 가져온다. 급여를 오름차순(ORDER BY 급여)으로 정렬하면 직전 행은 급여가 한 단계 작은 행이 되므로, LAG(급여) OVER (ORDER BY 급여)가 바로 직전급여를 반환한다.

선택지별 풀이

② ORDER BY를 DESC로 하면 정렬 방향이 뒤집혀, LAG가 가져오는 '이전 행'이 실제로는 급여가 더 큰 행이 되어 의도한 직전(작은쪽) 값이 아니다.

③ LEAD는 '다음(뒤) 행'의 값을 가져오므로 직전이 아니라 직후 급여가 출력된다.

④ LEAD + DESC는 방향까지 반대라 오름차순 기준의 직전 값과 다르다.

> **보충 개념** LAG(col, offset, default), LEAD(col, offset, default) 형태로 몇 행 앞/뒤를 볼지와 NULL 대체값을 지정할 수 있다.

13 2과목

부서 테이블의 부서명에는 'A연구소', '화학연구소', '연구소' 등의 값이 들어 있다. '연구소'로 끝나는 모든 부서명(글자 수와 무관하게, 'A연구소'·'화학연구소'·'연구소' 모두 포함)을 조회하는 WHERE 조건으로 옳은 것은?

① `WHERE 부서명 = '_연구소'`

② `WHERE 부서명 = '%연구소'`

③ `WHERE 부서명 LIKE '_연구소'`

④ `WHERE 부서명 LIKE '%연구소'`

정답 ④

핵심 해설

정답은 ④이다. LIKE '%연구소'에서 %는 0글자 이상의 임의 문자열을 의미하므로, 'A연구소'(앞 1글자), '화학연구소'(앞 2글자), '연구소'(앞 0글자)까지 모두 매칭된다. '연구소'로 끝나기만 하면 앞 글자 수와 무관하게 조회된다.

선택지별 풀이

① = '_연구소'는 등호 비교이므로 와일드카드가 동작하지 않고, 문자 그대로 '_연구소'라는 값과만 일치한다.

② = '%연구소' 역시 등호 비교라 '%'가 리터럴로 취급되어 문자 그대로 '%연구소'와만 일치한다. 와일드카드는 LIKE에서만 특수문자로 동작한다.

③ LIKE '_연구소'의 _는 '정확히 1글자'를 의미하므로 'A연구소'만 매칭되고, '화학연구소'(앞 2글자)나 '연구소'(앞 0글자)는 제외된다.

보충 개념 % = 0글자 이상, _ = 정확히 1글자. 부분 일치 검색은 반드시 = 이 아닌 LIKE와 함께 써야 한다.

14 2과목

다음 SQL의 실행 결과로 옳은 것은?

```
SELECT NVL2(NULL, 100, 200) AS 결과
FROM DUAL;
```

① 100 ② **200** ③ NULL ④ 실행 오류

정답 ②

핵심 해설

정답은 ②이다. NVL2(표현식, 참값, 거짓값)은 첫 번째 인자가 NULL이 '아니면' 두 번째 인자를, NULL '이면' 세 번째 인자를 반환한다. 여기서 첫 인자가 NULL이므로 세 번째 인자인 200이 반환된다.

선택지별 풀이

① 100은 첫 인자가 NULL이 아닐 때 반환되는 값이므로, 첫 인자가 NULL인 이 경우에는 나오지 않는다.

③ NVL2는 인자로 지정한 값을 반환하는 함수라서 결과가 NULL이 되지 않는다(반환 값 자체가 NULL로 지정된 경우가 아니라면).

④ NVL2는 정상적인 표준 함수 호출이므로 오류가 발생하지 않는다.

보충 개념 NVL(a, b)는 a가 NULL이면 b, NVL2(a, b, c)는 a가 NULL이 아니면 b·NULL이면 c를 돌려준다. 날짜·문자·숫자 등 다양한 타입에 쓸 수 있으며, EXTRACT 등 날짜 추출 함수와 결합해 'NULL이면 기본값, 아니면 연도 추출' 같은 처리에도 활용된다.

15 2과목

사원 테이블은 아래와 같이 각 사원의 상사(관리자)를 상사번호 컬럼으로 참조한다. 각 사원의 이름과 그 사원의 '상사 이름'을 함께 출력하려고 할 때, 조인 조건으로 옳은 것은?

컬럼	설명
사원번호	사원의 고유 번호(PK)
사원명	사원 이름
상사번호	그 사원의 상사(사원번호를 참조)

```
SELECT e.사원명 AS 사원, m.사원명 AS 상사
FROM 사원 e JOIN 사원 m
ON (㉠);
```

① e.사원번호 = m.사원번호 ② **e.상사번호 = m.사원번호**

③ e.사원번호 = m.상사번호 ④ e.상사번호 = m.상사번호

정답 ②

핵심 해설

정답은 ②이다. 한 테이블 안에서 자기 자신을 다시 참조하는 셀프 조인이다. e를 '사원', m을 '상사'로 두면, 사원(e)의 상사번호가 상사(m)의 사원번호와 같아야 그 상사 행이 매칭된다. 따라서 e.상사번호 = m.사원번호가 옳다.

선택지별 풀이

① e.사원번호 = m.사원번호는 같은 행끼리 매칭되어 자기 자신을 상사로 붙일 뿐 상사 정보가 나오지 않는다.

③ e.사원번호 = m.상사번호는 반대로 '내가 상사인 부하'를 찾는 조건이 되어 의도(내 상사 찾기)와 어긋난다.

④ e.상사번호 = m.상사번호는 '같은 상사를 둔 동료'를 묶는 조건으로, 상사 이름을 붙이는 것과 무관하다.

보충 개념 셀프 조인은 반드시 서로 다른 별칭(e, m)으로 같은 테이블을 두 번 참조한다. 상사가 없는 최상위 사원까지 보이려면 LEFT OUTER JOIN을 쓴다.

16 2과목

다음 Oracle 전용 표기(+)로 작성된 SQL과 동일한 결과를 내는 ANSI 표준 조인으로 옳은 것은?

```
SELECT *
FROM 사원 e, 부서 d
WHERE e.부서번호 = d.부서번호(+);
```

① 사원 e LEFT OUTER JOIN 부서 d ON e.부서번호 = d.부서번호

② 사원 e RIGHT OUTER JOIN 부서 d ON e.부서번호 = d.부서번호

③ 사원 e INNER JOIN 부서 d ON e.부서번호 = d.부서번호

④ 사원 e FULL OUTER JOIN 부서 d ON e.부서번호 = d.부서번호

정답 ①

핵심 해설

정답은 ①이다. Oracle의 (+) 표기는 '부족한 쪽(NULL을 채워 보존되지 않는 쪽)'에 붙인다. 여기서 (+)가 d.부서번호(부서 쪽)에 붙어 있으므로, 부서가 없어도 사원은 모두 보존된다. 즉 사원이 기준(왼쪽)으로 남는 LEFT OUTER JOIN이다.

선택지별 풀이

② RIGHT OUTER JOIN은 부서를 모두 보존하므로 (+)가 e 쪽에 붙은 경우와 같아 반대다.

③ INNER JOIN은 양쪽 모두 매칭되는 행만 남겨, 부서가 없는 사원이 사라지므로 (+) 의미를 잃는다.

④ FULL OUTER JOIN은 양쪽 모두 보존해 (+)가 한쪽에만 있는 경우와 결과가 다르다.

> **보충 개념** Oracle에서 (+)는 옵셔널(보존 안 되는) 테이블 쪽에 붙는다. '(+)가 붙은 반대편 테이블이 기준으로 살아남는다'로 기억하면 ANSI 변환이 쉽다.

17 2과목

학생(학번, 이름)과 수강(학번, 과목) 테이블이 있다. '강의를 수강하는(수강 기록이 있는) 학생'을 조회하려고 할 때, 그 방법으로 적절하지 않은 것은?

① `SELECT * FROM 학생 s WHERE EXISTS (SELECT 1 FROM 수강 c WHERE c.학번 = s.학번)`

② `SELECT * FROM 학생 s WHERE s.학번 IN (SELECT 학번 FROM 수강)`

③ `SELECT 학번 FROM 학생 INTERSECT SELECT 학번 FROM 수강`

④ `SELECT * FROM 학생 s WHERE NOT EXISTS (SELECT 1 FROM 수강 c WHERE c.학번 = s.학번)`

정답 ④

핵심 해설

정답은 ④이다(적절하지 않은 것).

④ NOT EXISTS는 수강 테이블에 해당 학번이 '존재하지 않는' 학생, 즉 강의를 하나도 수강하지 않는 학생을 조회한다. 문제 의도(수강하는 학생)와 정반대이므로 적절하지 않다. (모두 적절한 방법)

선택지별 풀이

① EXISTS는 수강 테이블에 같은 학번 행이 존재하는 학생만 남기므로 수강생을 정확히 조회한다.

② IN 서브쿼리는 수강 테이블에 나타난 학번 집합에 포함되는 학생을 반환하므로 EXISTS와 사실상 동일한 결과다.

③ INTERSECT는 학생의 학번과 수강의 학번의 교집합, 즉 양쪽 모두에 존재하는 학번(=수강한 학생)을 반환한다.

> **보충 개념** '존재하면' EXISTS/IN/INTERSECT, '존재하지 않으면' NOT EXISTS/NOT IN/MINUS(EXCEPT)로 매핑해 기억하면 유형을 빠르게 구분할 수 있다.

18 2과목

Oracle에서 사원 테이블 중 '사원번호가 가장 큰 사원 1명'을 조회하려고 한다. 올바른 SQL은?

① `SELECT * FROM (SELECT * FROM 사원 ORDER BY 사원번호 DESC) WHERE ROWNUM = 1`

② `SELECT TOP 1 * FROM 사원 ORDER BY 사원번호 DESC`

③ `SELECT * FROM 사원 WHERE ROWNUM = 1 ORDER BY 사원번호 DESC`

④ `SELECT * FROM 사원 WHERE 사원번호 = MAX(사원번호)`

정답 ①

핵심 해설

정답은 ①이다. ROWNUM은 정렬(ORDER BY)보다 먼저 매겨지므로, 먼저 인라인 뷰에서 사원번호 DESC로 정렬을 끝낸 뒤 바깥에서 ROWNUM = 1로 첫 행을 골라야 '가장 큰 사원번호 1건'을 정확히 얻는다.

선택지별 풀이

② TOP 절은 SQL Server(및 일부 DBMS) 문법으로, Oracle에서는 지원하지 않아 문법 오류가 난다.

③ ROWNUM은 ORDER BY가 적용되기 전에 부여되므로, 정렬 전 임의의 1행이 먼저 뽑힌 뒤 정렬되어 '최대 사원번호'가 아닌 엉뚱한 행이 나온다.

④ 집계 함수 MAX는 WHERE 절에서 직접 사용할 수 없다(집계 결과 비교는 서브쿼리나 HAVING로 해야 함). 문법 오류다.

> **보충 개념** Oracle 12c 이상에서는 ORDER BY 사원번호 DESC FETCH FIRST 1 ROW ONLY로도 같은 결과를 낼 수 있다. 핵심은 'ROWNUM은 정렬 이후가 아니라 이전에 매겨진다'는 실행 순서다.

19 2과목

비어 있는 주문 테이블에 대해 다음 명령을 순서대로 실행했다. 마지막 SELECT의 결과로 옳은 것은? (SAVEPOINT는 선언하지 않았다.)

```
INSERT INTO 주문 ...; -- 30건 삽입
TRUNCATE TABLE 임시로그; -- (임시로그 테이블 비우기, DDL)
INSERT INTO 주문 ...; -- 10건 추가 삽입
ROLLBACK;
SELECT COUNT(*) FROM 주문;
```

① 0 ② 15 **③ 30** ④ NULL

정답 ③

핵심 해설

정답은 ③ 30이다. TRUNCATE는 DDL이므로 실행 시점에 '암시적 커밋(implicit commit)'이 일어난다. 따라서 그 앞에서 주문에 넣은 30건은 TRUNCATE가 실행되는 순간 자동으로 커밋되어 확정된다. 이후 추가한 10건은 커밋되지 않은 상태에서 ROLLBACK되어 취소되므로, 최종적으로 주문에는 커밋된 30건만 남는다.

선택지별 풀이

① 0은 앞선 30건까지 롤백된 경우인데, DDL의 암시적 커밋 때문에 30건은 이미 확정되어 롤백 대상이 아니다.

② 15는 근거가 되는 연산이 없다(함정용 숫자).

④ COUNT(*)는 행이 0건이어도 0을 반환하지 NULL을 반환하지 않는다.

> **보충 개념** DDL(CREATE/ALTER/DROP/TRUNCATE)은 실행 전에 진행 중이던 트랜잭션을 자동 커밋한다. 그래서 DDL 앞의 DML은 ROLLBACK으로 되돌릴 수 없다는 점이 핵심이다.

20 2과목

고객 테이블에는 고객번호(NOT NULL)와 집주소 컬럼이 있고, 일부 고객은 집주소가 입력되지 않아 NULL이다. '집주소가 입력되지 않은(NULL) 고객의 수'를 구하는 식으로 옳은 것은?

① `COUNT(*)`

② `COUNT(집주소)`

③ `COUNT(고객번호) - COUNT(집주소)`

④ `COUNT(*) - COUNT(고객번호)`

정답 ③

핵심 해설

정답은 ③이다. COUNT(컬럼)은 해당 컬럼이 NULL인 행을 세지 않는다. 고객번호는 NOT NULL이라 COUNT(고객번호)는 전체 고객 수와 같고, COUNT(집주소)는 집주소가 있는(NULL이 아닌) 고객 수다. 따라서 두 값을 빼면 집주소가 NULL인 고객 수가 나온다.

선택지별 풀이

① COUNT(*)는 NULL 여부와 무관하게 모든 행(전체 고객)을 세므로 집주소 없는 고객만 세지 못한다.

② COUNT(집주소)는 오히려 집주소가 '있는' 고객 수라 정반대다.

④ 고객번호가 NOT NULL이면 COUNT(*)와 COUNT(고객번호)가 같아 결과가 항상 0이 되어 의미가 없다.

> **보충 개념** COUNT(*)는 NULL 포함 전체 행, COUNT(컬럼)은 그 컬럼의 NOT NULL 행만 센다. 이 차이를 이용해 '전체 - 값이 있는 수 = NULL의 수'를 계산할 수 있다.

21 2과목

다음 중 SQL 문(SELECT)의 논리적 실행 순서를 바르게 나열한 것은?

① SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY

② FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

③ FROM → GROUP BY → WHERE → HAVING → SELECT → ORDER BY

④ FROM → WHERE → HAVING → GROUP BY → SELECT → ORDER BY

정답 ②

핵심 해설

정답은 ②이다. SQL의 논리적 실행 순서는 FROM(테이블 확정) → WHERE(행 필터) → GROUP BY(그룹화) → HAVING(그룹 필터) → SELECT(컬럼 선택·별칭) → ORDER BY(정렬)이다.

선택지별 풀이

① SELECT가 맨 앞이라 틀림(SELECT는 거의 마지막).

③ WHERE가 GROUP BY 뒤라 틀림(행 필터가 그룹화보다 먼저).

④ HAVING이 GROUP BY보다 앞이라 틀림(HAVING은 그룹화 이후).

> **보충 개념** WHERE는 그룹화 전 개별 행을, HAVING은 그룹화 후 집계 결과를 필터링한다. SELECT의 컬럼 별칭을 WHERE·GROUP BY에서 못 쓰는 이유도 이 순서 때문이다.

22 2과목

전화번호 컬럼(전화번호)에서 '010'으로 시작하는 행만 조회하려고 한다. WHERE 조건으로 옳은 것은?

① REGEXP_LIKE(전화번호, '[^010]')

② REGEXP_LIKE(전화번호, '010$')

③ REGEXP_LIKE(전화번호, '^010')

④ REGEXP_LIKE(전화번호, '02')

정답 ③

핵심 해설

정답은 ③이다. 정규표현식에서 캐럿(^)은 문자열의 시작 위치를 나타내는 앵커이므로 '^010'은 '010'으로 시작하는 문자열에 매칭된다.

선택지별 풀이

① 대괄호 안의 ^([^...])는 '부정(제외)' 문자 클래스로, '[^010]'은 0 또는 1이 아닌 문자가 하나라도 있으면 매칭되어 의도와 정반대다.

② '$'는 문자열의 끝 앵커이므로 '010$'은 '010'으로 끝나는 문자열을 찾는다.

④ '02'는 문자열 어디에든 '02'가 포함되면 매칭되며 시작 위치를 강제하지 않는다.

보충 개념 대괄호 밖의 ^는 시작 앵커, 대괄호 안 첫 위치의 ^는 부정으로 의미가 완전히 다르다.

23 2과목

다음 SQL을 실행했을 때 (결과1, 결과2)로 옳은 것은?

```
SELECT
CASE WHEN REGEXP_LIKE('BCD', 'B.D') THEN 'BCD' ELSE 'NULL' END AS 결과1,
CASE WHEN REGEXP_LIKE('BCD', 'B\.D') THEN 'BCD' ELSE 'NULL' END AS 결과2
FROM DUAL;
```

① BCD, BCD ② NULL, BCD **③ BCD, NULL** ④ NULL, NULL

정답 ③

핵심 해설

정답은 ③이다. 정규표현식에서 점(.)은 임의의 한 문자를 의미하고, 역슬래시로 이스케이프한 '\.'는 리터럴 점 문자 그 자체를 의미한다. 결과1: 패턴 'B.D'는 B + (임의 한 문자) + D이므로 'BCD'(B-C-D)에 매칭되어 'BCD'를 반환한다. 결과2: 패턴 'B\.D'는 B + 실제 마침표 + D를 요구하는데 'BCD'에는 마침표가 없으므로 매칭되지 않아 'NULL'을 반환한다. 따라서 (BCD, NULL)이다.

선택지별 풀이

① 결과2가 매칭된다고 본 오답,
② 두 결과를 뒤바꾼 오답,
④ 둘 다 불일치로 본 오답이다.

보충 개념 특수문자를 문자 자체로 찾으려면 반드시 이스케이프해야 한다.

24 2과목

집합 연산자(UNION, UNION ALL, INTERSECT, MINUS)에 대한 설명으로 옳지 않은 것은?

① 두 SELECT 결과 컬럼의 데이터 타입이 상호 호환되더라도 반드시 완전히 동일한 데이터 타입으로 일치시켜야만 사용할 수 있다.

② UNION은 두 집합을 합친 뒤 중복 행을 제거하며, 이 과정에서 내부적으로 정렬이 수반될 수 있다.

③ UNION ALL은 중복 행을 제거하지 않으며 별도의 정렬 작업을 수행하지 않는다.

④ 집합 연산에 참여하는 각 SELECT 문의 컬럼 개수는 서로 같아야 한다.

정답 ①

핵심 해설

정답은 ①이다. 집합 연산자는 각 SELECT의 컬럼 개수가 같고 대응 컬럼의 데이터 타입이 '상호 호환(compatible)'되면 사용할 수 있으며, 반드시 완전히 동일한 타입일 필요는 없다(예: 숫자끼리, 문자끼리 호환되면 됨). '무조건 동일 타입이어야 한다'는 서술이 틀렸다.

선택지별 풀이

② UNION은 중복을 제거하며 이 과정에서 내부적으로 정렬(Sort Unique)이 수반될 수 있으므로 옳다(다만 항상 정렬된 결과를 보장하지는 않는다).
③ UNION ALL은 중복 제거와 정렬을 하지 않아 UNION보다 빠르므로 옳다.
④ 대응되는 컬럼 개수 일치는 집합 연산의 필수 조건이므로 옳다.

보충 개념 UNION은 정렬 비용이 있고, 중복 제거가 불필요하면 UNION ALL이 성능상 유리하다.

25 2과목

CTAS(CREATE TABLE ... AS SELECT)로 새 테이블을 생성할 때에 대한 설명으로 옳지 않은 것은?

① 원본 테이블의 기본키(PK)·외래키(FK) 제약조건이 새 테이블에 그대로 함께 복사된다.

② 원본에서 선택된 컬럼의 데이터 타입과 크기는 새 테이블에 전달된다.

③ 원본 컬럼에 설정된 NOT NULL 제약조건은 새 테이블에 전달된다.

④ SELECT 절에서 컬럼에 별칭(alias)을 부여하면 그 별칭이 새 테이블의 컬럼명이 된다.

정답 ①

핵심 해설

정답은 ①이다. CTAS는 SELECT 결과의 컬럼 구조와 데이터를 복사하지만 기본키(PK)·외래키(FK)·UNIQUE·CHECK 같은 제약조건은 승계되지 않는다. 즉 PK/FK는 복사되지 않으므로

선택지별 풀이

① **이 틀렸다.**

② 컬럼의 데이터 타입·크기는 그대로 전달되므로 옳다.

③ NOT NULL 제약조건은 CTAS로 함께 전달되는 유일한 제약이므로 옳다.

④ SELECT 절의 별칭은 생성되는 테이블의 컬럼명이 되므로(연산식·함수 컬럼은 별칭 필수) 옳다.

> **보충 개념** 'NOT NULL과 컬럼 정의는 전달, 나머지 제약(PK/FK/UNIQUE/CHECK)은 미전달'이 CTAS의 핵심 규칙이다.

---

26 2과목

다음과 같이 정의된 주문 테이블에 INSERT를 실행할 때, 오류가 발생하는 것은?

```
CREATE TABLE 주문 (
주문번호 NUMBER,
주문일자 DATE,
상품코드 CHAR(3)
);
```

① `INSERT INTO 주문 VALUES (1, TO_DATE('20240606','YYYYMMDD'), '001');`

② `INSERT INTO 주문 (주문번호, 상품코드) VALUES (2, '002');`

③ `INSERT INTO 주문 VALUES (3, SYSDATE, '003');`

④ `INSERT INTO 주문 VALUES (4, 20240606, '004');`

정답 ④

핵심 해설

정답은 ④다. 주문일자 컬럼은 DATE 타입인데 20240606은 숫자 리터럴이며, 오라클은 숫자를 날짜로 묵시적 변환하지 못하므로 오류(ORA-00932 등)가 발생한다. 날짜 값은 TO_DATE 등으로 변환하거나 날짜 리터럴을 사용해야 한다.

선택지별 풀이

① TO_DATE로 문자열을 DATE로 명시 변환했으므로 정상이다.

② 컬럼 리스트를 명시해 주문번호·상품코드만 넣고 주문일자는 NULL로 들어가므로 정상이다(NOT NULL 제약 없음).

③ SYSDATE는 DATE 타입이므로 정상이다.

> **보충 개념** VALUES에 컬럼 리스트를 생략하면 테이블 정의 순서·타입에 정확히 맞춰야 하며, 타입 불일치 시 묵시적 변환 가능 여부가 성패를 가른다.

---

27 2과목

주문일자 순서대로 금액의 '누적 합계'를 구하려고 한다. 빈칸에 들어갈 윈도우 절로 옳은 것은?

```
SELECT 주문일자, 금액,
SUM(금액) OVER (ORDER BY 주문일자
```

```
_____ ) AS 누적금액
FROM 주문;
```

① ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING

② RANGE BETWEEN UNBOUNDED FOLLOWING AND CURRENT ROW

③ ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

④ ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING

정답 ③

핵심 해설

정답은 ③이다. 첫 행부터 현재 행까지의 누적 합계는 프레임을 'UNBOUNDED PRECEDING(맨 처음) ~ CURRENT ROW(현재 행)'로 지정하면 얻는다.

선택지별 풀이

① 'CURRENT ROW ~ UNBOUNDED FOLLOWING'은 현재 행부터 끝까지를 더하는 역방향 누적이라 의도와 다르다.

② 'UNBOUNDED FOLLOWING'을 프레임 시작으로 쓰는 것은 문법 오류다(시작 경계에 FOLLOWING 무한을 둘 수 없음).

④ '1 PRECEDING ~ 1 FOLLOWING'은 앞뒤 한 행씩만 포함하는 이동 구간이라 누적이 아니다.

> **보충 개념** ORDER BY만 쓰고 프레임을 생략하면 기본값이 'RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW'라 역시 누적이 되지만, 동일 정렬값 동점 처리에서 ROWS와 차이가 날 수 있다.

28 2과목

주문일시(DATE, 시·분·초 포함) 컬럼에서 '2025년 10월 20일 하루 전체'에 해당하는 행을 조회하려고 한다. 조회 조건으로 옳지 않은 것은?

① 주문일시 >= TO_DATE('2025-10-20','YYYY-MM-DD') AND 주문일시 < TO_DATE('2025-10-21','YYYY-MM-DD')

② TRUNC(주문일시) = TO_DATE('2025-10-20','YYYY-MM-DD')

③ 주문일시 BETWEEN TO_DATE('2025-10-20','YYYY-MM-DD') AND TO_DATE('2025-10-21','YYYY-MM-DD')

④ 주문일시 >= TO_DATE('2025-10-20','YYYY-MM-DD') AND 주문일시 <= TO_DATE('2025-10-20','YYYY-MM-DD') + 1 - 1/86400

정답 ③

핵심 해설

정답은 ③이다. DATE에는 시·분·초가 포함되므로 TO_DATE('2025-10-20')은 실제로 '2025-10-20 00:00:00'을 의미한다. BETWEEN은 양 끝 경계를 포함하므로

선택지별 풀이

③ 은 상한이 '2025-10-21 00:00:00'까지 포함되어, 10월 21일 자정(00:00:00) 주문까지 잘못 조회한다.

① 20일 0시 이상이면서 21일 0시 '미만'이라 하루를 정확히 담는 표준 방식이다.

② TRUNC로 시분초를 잘라 날짜만 비교하므로 하루 전체가 정확히 걸린다.

④ 상한을 '20일 0시 + 1일 - 1초 = 20일 23:59:59'로 잡아 21일 자정을 배제하므로 옳다.

> **보충 개념** 날짜 범위 조회에서 BETWEEN은 경계 포함 특성 때문에 다음날 자정 유입 위험이 있어 '>= 시작 AND < 다음날' 패턴이 안전하다.

29 2과목

다음과 같은 계층형 질의(START WITH ... CONNECT BY)에 대한 설명으로 옳지 않은 것은?

```
SELECT LEVEL, 사원명, 관리자ID
FROM 사원
```

```
START WITH 관리자ID IS NULL
CONNECT BY PRIOR 사원ID = 관리자ID;
```

① START WITH는 계층 전개를 시작할 루트(최상위) 행의 조건을 지정한다.

② CONNECT BY PRIOR는 부모 행과 자식 행을 연결하는 전개 방향을 결정한다.

③ LEVEL은 루트를 1로 하여 아래로 내려갈수록 1씩 증가하는 계층의 깊이를 반환한다.

④ ORDER SIBLINGS BY 대신 일반 ORDER BY를 사용해도 형제 노드 간 정렬은 물론 계층 구조 자체는 항상 그대로 유지된다.

정답 ④

핵심 해설

정답은 ④다. 계층형 질의 결과를 일반 ORDER BY로 정렬하면 전체 행이 정렬 기준대로 재배열되어 부모-자식 계층 순서(전개 순서)가 깨질 수 있다. 계층 구조를 유지한 채 형제(SIBLINGS) 노드만 정렬하려면 'ORDER SIBLINGS BY'를 써야 한다.

선택지별 풀이

① START WITH는 전개의 시작점(루트) 조건을 지정하므로 옳다.

② CONNECT BY PRIOR는 PRIOR가 어느 쪽(부모/자식)을 가리키는지로 순방향/역방향 전개를 결정하므로 옳다.

③ LEVEL은 루트가 1이며 자식으로 내려갈수록 1씩 증가하는 깊이 값이므로 옳다.

> **보충 개념** 계층 순서 유지 정렬은 ORDER BY가 아니라 ORDER SIBLINGS BY라는 점이 자주 출제된다.

30 2과목

WHERE 절에서 논리 연산자 AND, OR, NOT이 함께 사용될 때의 평가 순서에 대한 설명으로 옳은 것은?

① 같은 수준에서 괄호가 없으면 AND가 OR보다 먼저 평가된다.

② 괄호가 없으면 OR가 AND보다 먼저 평가된다.

③ 논리 연산자는 우선순위 없이 항상 왼쪽에서 오른쪽 순서로만 평가된다.

④ NOT은 AND, OR가 모두 평가된 뒤 가장 마지막에 평가된다.

정답 ①

핵심 해설

정답은 ①이다. 논리 연산자의 우선순위는 NOT > AND > OR 순이므로, 괄호가 없으면 AND가 OR보다 먼저 평가된다. 예: 'A OR B AND C'는 'A OR (B AND C)'로 해석된다.

선택지별 풀이

② OR가 AND보다 먼저라는 것은 우선순위를 뒤집은 틀린 설명이다.

③ 논리 연산자에는 분명한 우선순위(NOT>AND>OR)가 있으므로 '무조건 좌→우'는 틀리다.

④ NOT은 세 연산자 중 가장 먼저 평가되므로 '가장 마지막'은 틀리다.

> **보충 개념** 의도한 조건을 확실히 하려면 괄호로 우선순위를 명시하는 것이 안전하며, AND/OR 혼용 시 괄호 누락은 대표적인 논리 오류 원인이다.

31 2과목

다음 SQL에 대한 설명으로 옳은 것은?

```
SELECT 사원명, 급여
FROM 사원
WHERE 급여 > ALL (SELECT MAX(급여)
FROM 사원
GROUP BY 부서번호);
```

① 서브쿼리가 반환하는 부서가 하나뿐이라면, 단일 값과 비교하는 것과 결과가 같다.

② 서브쿼리가 여러 부서(여러 행)를 반환하면 단일행 비교 오류가 발생한다.

③ ALL을 ANY로 바꿔도 결과가 항상 동일하다.

④ ALL 대신 등호(=)를 사용해도 결과가 동일하다.

정답 ①

핵심 해설

> ALL (집합)은 집합의 '모든' 값보다 커야 하므로, 사실상 부서별 최고 급여들 중 가장 큰 값(전체 최댓값)보다 큰 사원을 조회한다.

선택지별 풀이

① **서브쿼리가 부서 하나만 반환하면 값이 하나뿐이라 단일 값 비교와 동일한 결과가 된다 → 옳음(정답).**

② ALL·ANY 같은 다중행 연산자는 여러 행을 정상적으로 처리하도록 만들어진 연산자라 오류가 나지 않는다 → 틀림.

③ > ALL은 최댓값보다 큰 것, > ANY는 최솟값보다 큰 것을 의미해 결과가 서로 다르다 → 틀림.

④ =는 단일행 연산자여서 서브쿼리가 여러 행을 반환하면 'single-row subquery returns more than one row' 오류가 발생하므로 동일하지 않다 → 틀림.

> **보충 개념** 다중행 서브쿼리 연산자는 IN, ANY(=SOME), ALL, EXISTS이며, >ALL(max집합)은 전체 최댓값 초과, >ANY(max집합)은 최솟값 초과와 같다.

32 2과목

CUBE, ROLLUP, GROUPING SETS에 대한 설명으로 옳은 것은?

① ROLLUP(부서, 직급)은 (부서,직급), (부서), (직급), () 의 4가지 집계를 생성한다.

② CUBE(부서, 직급)은 (부서,직급), (부서), () 의 3가지 집계만 생성한다.

③ GROUPING SETS는 명시한 그룹 외에 전체 합계 ()를 항상 자동으로 포함한다.

④ GROUPING SETS(부서, 직급)은 (부서), (직급) 두 그룹만 생성하며 UNION ALL로 표현할 수 있다.

정답 ④

핵심 해설

④ GROUPING SETS는 나열한 집계 단위만 정확히 생성한다. GROUPING SETS(부서, 직급)은 (부서)별 집계와 (직급)별 집계 두 결과를 각각 만들어 붙인 것과 같고, GROUP BY 부서 UNION ALL GROUP BY 직급으로 표현 가능하다 → 옳음(정답).

선택지별 풀이

① ROLLUP(부서,직급)은 오른쪽부터 계층적으로 제거해 (부서,직급),(부서),() 의 3가지만 생성한다. (직급) 단독은 없다 → 틀림.

② CUBE(부서,직급)은 가능한 모든 조합 (부서,직급),(부서),(직급),() 4가지를 생성한다 → 틀림.

③ GROUPING SETS는 지정한 집계만 만들며 ()(전체합계)를 원하면 명시적으로 () 를 넣어야 한다 → 틀림.

> **보충 개념** ROLLUP은 n+1개, CUBE는 2^n개 집계 그룹을 만들고, GROUPING SETS는 원하는 조합만 골라 지정한다.

33 2과목

다음 Round 함수 결과에 대한 설명 중 옳은 것을 모두 고른 것은?

가. Round(136.2, -1) = 140

나. Round(-7.532, 2) = -7.5

다. Round(3.561, 0) = 3

① 가 ② 나 ③ 가, 나 ④ 나, 다

정답 ①

핵심 해설

정답은 ① 가이다. 가. Round(136.2, -1)은 일의 자리에서 반올림해 140이다 → 옳다. 나. Round(-7.532, 2)는 소수 셋째 자리(2)에서 반올림해 -7.53이다. -7.5가 아니므로 틀리다. 다. Round(3.561, 0)은 소수 첫째 자리에서 반올림해 4이다. 3이 아니므로 틀리다. 따라서 옳은 것은 (가)뿐이다.

> **보충 개념** Round(수, n)에서 n이 양수면 소수점 이하 n자리까지, n이 0이면 정수로, n이 음수면 정수부 |n|자리에서 반올림한다.

34 2과목

다음 중 현재 날짜에서 '월(月)'을 추출하지 않는 것은?

① EXTRACT(YEAR FROM SYSDATE)

② EXTRACT(MONTH FROM SYSDATE)

③ TO_CHAR(SYSDATE, 'MM')

④ TO_NUMBER(TO_CHAR(SYSDATE, 'MM'))

정답 ①

핵심 해설

① EXTRACT(YEAR FROM SYSDATE)는 '연도'를 추출하므로 월을 반환하지 않는다 → 정답.

선택지별 풀이

② EXTRACT(MONTH FROM SYSDATE)는 월을 숫자로 추출한다.

③ TO_CHAR(SYSDATE, 'MM')은 월을 두 자리 문자열('07')로 반환한다.

④ TO_NUMBER(TO_CHAR(SYSDATE,'MM'))은 그 문자열을 숫자(7)로 변환한 월이다.

> **보충 개념** EXTRACT는 날짜/타임스탬프에서 YEAR·MONTH·DAY·HOUR 등 특정 필드를 숫자로 뽑아내는 표준 함수다. 월을 얻는 대표 방법은 EXTRACT(MONTH FROM ...)과 TO_CHAR(...,'MM')이다.

35 2과목

다음 SELECT의 결과를 순서대로 바르게 나열한 것은?

사원

부서	성과급
A	(NULL)
A	(NULL)
B	10
B	20
C	30
C	30

```
SELECT SUM(성과급)
FROM 사원
GROUP BY 부서
ORDER BY 부서;
```

① NULL, 0, 60 **② NULL, 30, 60** ③ 0, 30, 60 ④ 60, 30, NULL

정답 ②

핵심 해설

집계 함수 SUM은 NULL을 계산에서 제외한다. 그러나 그룹의 모든 값이 NULL이면 더할 값 자체가 없으므로 결과는 0이 아니라 NULL이 된다. A: 두 행 모두 NULL → 합계 대상 없음 → NULL B: 10 + 20 = 30 C: 30 + 30 = 60 부서 순으로 NULL, 30, 60 → 정답

선택지별 풀이

② .

① A그룹을 0으로 본 오답(SUM은 전부 NULL이면 0이 아닌 NULL).

③ 역시 NULL을 0으로 오인.

④ 순서를 뒤집은 오답.

> **보충 개념** COUNT(*)는 NULL도 세어 A그룹이 2가 되지만, COUNT(성과급)·SUM·AVG는 NULL을 제외한다. 특히 전부 NULL인 그룹의 SUM/AVG/MAX/MIN은 NULL이다.

36 2과목

다음 SQL의 결과를 순서대로 바르게 나열한 것은? (T1, T2, T3의 열 이름은 모두 코드)

T1

코드
A
B

T2

코드
B

T3

코드
B
B
C

```
SELECT 코드 FROM T1
MINUS
SELECT 코드 FROM T2
UNION ALL
SELECT 코드 FROM T3;
```

① A, B, C ② A, A, B, C **③ A, B, B, C** ④ B, B, C

정답 ③

핵심 해설

집합 연산자는 우선순위가 같아 위에서 아래로(왼쪽에서 오른쪽으로) 순서대로 평가된다. 따라서 (T1 MINUS T2) 를 먼저 계산하고 그 결과에 UNION ALL T3를 한다. T1 MINUS T2 = {A, B} - {B} = {A} (MINUS는 차집합이며 중복을 제거한다) {A} UNION ALL T3 = A, (B, B, C) = A, B, B, C UNION ALL은 중복을 제거하지 않고 그대로 이어 붙이므로 B가 두 번 남는다 → 정답

선택지별 풀이

③ .

① 최종 연산을 UNION(중복 제거)으로 오인한 오답.

② A가 두 번이 될 근거 없음.

④ MINUS 결과 {A}를 누락한 오답.

> **보충 개념** MINUS·INTERSECT·UNION은 중복을 제거하지만 UNION ALL만 중복을 유지한다. 우선순위를 바꾸려면 괄호를 사용한다.

37 2과목

다음 SQL에 대한 설명으로 옳은 것은?

```
SELECT 배우명, 출연료
FROM 출연 O
WHERE 출연료 > (SELECT AVG(출연료)
FROM 출연 I
WHERE I.영화ID = O.영화ID);
```

① 서브쿼리가 메인쿼리와 무관하게 딱 한 번만 실행되는 비연관 서브쿼리이다.

② 서브쿼리를 먼저 한 번 실행해 상수 값을 구한 뒤 메인쿼리를 실행한다.

③ 서브쿼리에서는 메인쿼리의 컬럼을 참조할 수 없다.

④ 메인쿼리의 각 행마다 서브쿼리가 실행되어 값을 비교하는 연관(상관) 서브쿼리이다.

정답 ④

핵심 해설

서브쿼리 내부에서 메인쿼리의 별칭 O의 컬럼(O.영화ID)을 참조하고 있으므로, 이 서브쿼리는 메인쿼리의 행에 의존해 실행되는 연관(상관) 서브쿼리다.

선택지별 풀이

④ 메인쿼리가 읽는 각 배우 행마다 그 배우가 속한 영화의 평균 출연료를 다시 계산해 비교한다 → 옳음(정답). 즉 '같은 영화 안에서 평균보다 많이 받은 배우'를 찾는다.

① 연관 서브쿼리는 메인 행마다 반복 실행되므로 '한 번만' 실행되지 않는다 → 틀림.

② 비연관 서브쿼리에 대한 설명이다. 이 쿼리는 O.영화ID 때문에 먼저 독립 실행할 수 없다 → 틀림.

③ 연관 서브쿼리의 핵심이 바로 메인쿼리 컬럼 참조이므로 틀림.

> **보충 개념** 연관 서브쿼리는 EXISTS/NOT EXISTS와 자주 쓰이며, 외부 행마다 조건을 확인하는 방식으로 동작한다.

---

38 2과목

다음 쿼리에서 '박'의 순위와, RANK를 DENSE_RANK로 바꿨을 때 '박'의 순위를 바르게 짝지은 것은?

성적

이름	점수
김	90
이	90
박	80
최	70

```
SELECT 이름,
RANK() OVER (ORDER BY 점수 DESC) AS 순위
FROM 성적;
```

① 3, 3 **② 3, 2** ③ 4, 2 ④ 4, 3

정답 ②

핵심 해설

RANK()는 동순위가 있으면 그 수만큼 다음 순위를 건너뛴다(공동 1위가 둘이면 다음은 3위). DENSE_RANK()는 동순위가 있어도 다음 순위를 건너뛰지 않는다. 점수 내림차순: 김90 · 이90(공동 1위), 박80, 최70 RANK: 김=1, 이=1, 박=3(2를 건너뜀), 최=4 → 박은 3 DENSE_RANK: 김=1, 이=1, 박=2, 최=3 → 박은 2 따라서 (3, 2) → 정답 ②.

① DENSE_RANK가 건너뛴다고 오인. ③④ RANK에서 박을 4로 오인(80보다 위는 90 두 명뿐이므로 3위).

보충 개념 ROW_NUMBER()는 동점이어도 임의로 유일한 번호(1,2,3,4)를 부여한다. 세 함수 모두 ORDER BY가 필수인 순위 윈도우 함수다.

39 2과목

다음 계층형 질의의 동작으로 옳은 것은? (parent_id는 상위 category_id를 가리킨다)

```
SELECT category_id, parent_id, LEVEL
FROM category
START WITH category_id = 11
CONNECT BY category_id = PRIOR parent_id;
```

① category_id 11에서 시작해 하위(자식) 방향으로 전개한다.

② category_id 2에서 시작해 하위(자식) 방향으로 전개한다.

③ category_id 11에서 시작해 정방향(부모→자식)으로 전개한다.

④ category_id 11에서 시작해 역방향(자식→부모, 상위)으로 전개한다.

정답 ④

핵심 해설

START WITH가 category_id = 11이므로 전개는 11번 행에서 시작한다. CONNECT BY 조건이 category_id = PRIOR parent_id인 점이 핵심이다. PRIOR가 붙은 쪽(parent_id)이 '이전(상위 단계) 행'의 값이고, 다음에 찾을 행의 조건은 category_id = 이전 행의 parent_id다. 즉 현재 노드의 부모를 다음 노드로 찾아 올라가는 역방향(자식→부모, 상위) 전개다. 따라서 11에서 시작해 그 부모, 다시 그 부모… 로 상위 계층을 따라 올라간다 → 정답

선택지별 풀이

④ .

① 방향이 반대다(이 조건은 자식이 아니라 부모를 찾는다).

② 시작점이 11이지 2가 아니다.

③ 정방향이라면 PRIOR category_id = parent_id 형태여야 한다.

보충 개념 정방향(부모→자식)은 CONNECT BY PRIOR category_id = parent_id, 역방향(자식→부모)은 CONNECT BY category_id = PRIOR parent_id로 PRIOR의 위치가 방향을 결정한다.

40 2과목

NATURAL JOIN에 대한 설명으로 옳은 것은?

① 조인에 사용되는 공통 컬럼에는 반드시 테이블명(별칭) 접두사를 붙여야 한다.

② 두 테이블에 이름이 같은 컬럼이 하나도 없어도 조인 조건이 자동으로 만들어진다.

③ NATURAL JOIN과 USING 절을 함께 사용하면 오류가 발생한다.

④ 컬럼 이름만 같으면 데이터 타입이 달라도 자동으로 조인된다.

정답 ③

핵심 해설

③ NATURAL JOIN은 조인 조건을 자동으로 결정하는 방식이고 USING은 조인 컬럼을 명시하는 방식이므로, 둘을 함께 쓰면 문법 오류가 발생한다 → 옳음(정답).

선택지별 풀이

① NATURAL JOIN(및 USING)으로 묶인 공통 컬럼은 오히려 테이블 접두사를 붙이면 안 된다. 접두사를 붙이면 오류가 난다 → 틀림.

② 이름이 같은 컬럼이 하나도 없으면 조인 조건이 만들어지지 않아 사실상 카티션 곱(교차 조인)이 된다 → 틀림.

④ 이름이 같아도 데이터 타입이 호환되지 않으면 오류가 발생한다. 컬럼명만으로 무조건 조인되는 것은 아니다 → 틀림.

보충 개념 NATURAL JOIN은 같은 이름의 모든 컬럼을 자동으로 등가 조인하며, 결과에서 공통 컬럼은 한 번만 표시된다. 조인 컬럼을 통제하고 싶으면 USING 또는 ON을 쓴다.

41 2과목

트랜잭션(Transaction)과 TCL에 대한 다음 설명 중 옳은 것을 모두 고른 것은?

가. DDL 문장을 실행하면 묵시적으로 COMMIT이 수행된다.
나. 트랜잭션은 데이터베이스의 논리적 연산(작업) 단위이다.
다. SAVEPOINT를 지정하면 해당 지점에서 트랜잭션 전체가 즉시 종료된다.

① 가 ② 다 ③ 나, 다 **④ 가, 나**

정답 ④

핵심 해설

가(참): CREATE·ALTER·DROP 같은 DDL은 실행 직전·직후에 묵시적 COMMIT이 일어나므로, DDL 앞의 미완료 DML까지 함께 확정된다. 나(참): 트랜잭션은 '더 이상 나눌 수 없는 논리적 연산 단위'로 정의되며 원자성(Atomicity)을 보장한다. 다(거짓): SAVEPOINT는 트랜잭션 내에 되돌아갈 지점을 표시할 뿐 트랜잭션을 종료시키지 않는다. 이후 ROLLBACK TO 해당지점 을 실행하면 그 지점 이후 변경만 취소되고 트랜잭션은 계속 유지된다. 따라서 옳은 것은 가, 나 이므로 정답은 ④.

보충 개념 트랜잭션 종료는 COMMIT 또는 (전체)ROLLBACK으로만 이루어진다.

42 2과목

다음 주문 테이블에 대해 아래 SQL을 실행할 때 결과로 반환되는 '건수' 값들을 모두 고른 것은?

고객	배송일자
A	2024-01-01
A	2024-01-03
A	2024-01-05
B	2024-02-01
B	2024-02-10
B	(NULL)
C	2024-03-01

```
SELECT 고객, COUNT(배송일자) AS 건수
FROM 주문
GROUP BY 고객
HAVING COUNT(배송일자) >= 2;
```

① 3 **② 2, 3** ③ 1, 2, 3 ④ 3, 4

정답 ②

핵심 해설

COUNT(컬럼)은 해당 컬럼이 NULL인 행을 세지 않는다. 고객별 COUNT(배송일자)는 A=3, B=2(마지막 NULL 제외), C=1이다. HAVING COUNT(배송일자) >= 2 조건으로 A(3), B(2)만 통과하고 C(1)는 걸러진다. 따라서 반환되는 건수 값은 2와 3이므로 정답은 ②. 오답: ①은 B가 누락, ③은 필터되는 C(1)를 포함, ④의 4는 어떤 고객에도 없는 값이다.

보충 개념 COUNT(*)였다면 B는 3이 되어 결과가 달라진다 — COUNT(*)와 COUNT(컬럼)의 NULL 처리 차이가 핵심이다.

43 2과목

다음 SQL의 실행 결과로 옳은 것은? 사원(사원명, 보너스): ('김', NULL), ('이', 50)

```
SELECT 사원명,
NVL2(보너스, 보너스 + 100, 0) AS 지급액
FROM 사원;
```

① 김 = 100, 이 = 150
② 김 = 0, 이 = 50
③ 김 = 0, 이 = 150
④ 김 = NULL, 이 = 150

정답 ③

핵심 해설

NVL2(식, 값1, 값2)는 첫 인자 '식'이 NULL이 아니면 값1을, NULL이면 값2를 반환한다. 김은 보너스가 NULL이므로 값2인 0이 반환된다. 이는 보너스가 50(NOT NULL)이므로 값1인 '보너스 + 100' = 150이 반환된다. 따라서 김=0, 이=150 으로 정답은 ③. 오답: ①은 김을 NULL이 아닌 것처럼 계산, ②는 값1을 잘못 적용, ④는 NVL2가 NULL을 그대로 반환한다는 오해다.

> **보충 개념** NVL(식, 대체값)은 인자가 2개(NULL일 때만 치환), NVL2는 인자가 3개로 NULL 여부에 따라 서로 다른 값을 돌려준다는 점을 구분해야 한다.

44 2과목

Oracle 계층형 질의(Hierarchical Query)에 대한 설명으로 옳지 않은 것은?

① START WITH 절은 계층 구조 전개를 시작하는 루트(root) 행을 지정한다.
② LEVEL 의사컬럼은 시작 값을 임의로 지정할 수 있어 루트의 LEVEL이 반드시 1이 되는 것은 아니다.
③ 데이터에 순환(cycle)이 존재할 수 있는 경우 NOCYCLE 옵션으로 무한 전개를 방지할 수 있다.
④ ORDER SIBLINGS BY 절은 동일한 부모를 가진 형제(sibling) 노드 사이에서만 정렬을 수행한다.

정답 ②

핵심 해설

정답은 옳지 않은 진술인 ②. LEVEL은 계층 전개 깊이를 나타내는 의사컬럼으로, 루트 행은 항상 1에서 시작하고 한 단계 내려갈 때마다 1씩 증가한다. 사용자가 시작 값을 임의로 바꿀 수 없다. ①(참): START WITH가 전개의 시작점(루트)을 지정한다. ③(참): CONNECT BY NOCYCLE 을 사용하면 순환 데이터에서 무한 루프를 방지하고, CONNECT_BY_ISCYCLE로 순환 행을 식별할 수 있다. ④(참): ORDER SIBLINGS BY는 계층 구조를 깨지 않으면서 같은 부모 아래 형제 노드만 정렬한다.

> **보충 개념** 전개 방향은 CONNECT BY PRIOR의 위치로 결정된다 — PRIOR가 부모키 쪽이면 부모→자식(순방향) 전개다.

45 2과목

다음 두 테이블에 대해 아래 SQL을 실행할 때 반환되는 행의 수는? A(col): 1, 1, 2 B(col): 2, 3

```
SELECT col FROM A
UNION ALL
SELECT col FROM B;
```

① 5
② 4
③ 3
④ 2

정답 ①

핵심 해설

UNION ALL은 두 결과 집합을 중복 제거 없이 그대로 이어 붙이며 정렬도 수행하지 않는다. A의 3건(1,1,2)과 B의 2건(2,3)이 모두 유지되어 총 5건(1,1,2,2,3)이 반환되므로 정답은 ①. 오답: ②(4)는 존재하지 않는 계산, ③(3)은 UNION(중복 제거)을 적용했을 때의 결과 {1,2,3}, ④(2)는 교집합(INTERSECT) 착각이다.

> **보충 개념** UNION은 중복 제거 + 정렬 오버헤드가 있어 중복이 없음이 확실하거나 성능이 중요한 경우 UNION ALL을 사용한다.

46 2과목

`GROUP BY ROLLUP(부서, 직급)` 을 사용한 집계에 대한 설명으로 옳은 것은?

① (부서, 직급)별 집계와 함께 부서별 소계, 그리고 전체 합계 행이 생성된다.

② (부서, 직급)별 집계와 함께 직급별 소계도 함께 생성된다.

③ ROLLUP은 GROUP BY 없이 단독으로 사용할 수 있다.

④ ROLLUP(부서, 직급)과 CUBE(부서, 직급)의 결과 행 수는 항상 동일하다.

정답 ①

핵심 해설

ROLLUP(a, b)는 인자 순서에 따라 계층적 소계를 만든다: (부서, 직급) → (부서) → () 세 수준의 그룹을 생성하므로, 상세 집계 + 부서별 소계 + 전체 합계가 나온다. 따라서 정답은 ①. ②(거짓): 직급 단독 소계는 ROLLUP이 만들지 않는다 — 모든 조합을 만드는 CUBE에서만 생성된다. ③(거짓): ROLLUP은 GROUP BY의 확장 문법으로 GROUP BY 없이 단독 사용할 수 없다. ④(거짓): CUBE(a,b)는 (a,b),(a),(b),() 네 그룹으로 ROLLUP(세 그룹)보다 결과 행이 많거나 같다.

> **보충 개념** 어떤 소계 행인지 구분하려면 GROUPING 또는 GROUPING_ID 함수를 사용한다.

47 2과목

다음 테이블 T와 SQL에 대해, id 순으로 출력되는 '이동합' 컬럼의 값을 순서대로 바르게 나열한 것은? T(id, val): (1, 100), (2, 200), (3, 300), (4, 400)

```
SELECT val,
SUM(val) OVER (ORDER BY id
ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING) AS 이동합
FROM T
ORDER BY id;
```

① 100, 300, 600, 1000

② 300, 500, 700, 700

③ 300, 600, 900, 700

④ 100, 200, 300, 400

정답 ③

핵심 해설

ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING 은 현재 행 기준 바로 앞 1행 + 현재 행 + 바로 뒤 1행의 물리적 범위를 합산한다. id=1: 앞 행 없음 → 100+200=300. id=2: 100+200+300=600. id=3: 200+300+400=900. id=4: 뒤 행 없음 → 300+400=700. 따라서 300, 600, 900, 700 으로 정답은 ③. 오답: ①은 UNBOUNDED PRECEDING(누적합)의 결과, ④는 프레임을 무시하고 val을 그대로 둔 값이다.

> **보충 개념** 프레임 경계 밖의 행은 자동으로 제외되므로 첫·마지막 행에서는 합산 대상이 줄어든다. ROWS(물리적 행 수)와 RANGE(값 범위)의 차이도 함께 이해해야 한다.

48 2과목

다음 SQL의 실행 결과로 옳은 것은? 사원(사원번호): 1, 2, 3 관리(관리자번호): 1, NULL

```
SELECT 사원번호
FROM 사원
WHERE 사원번호 NOT IN (SELECT 관리자번호 FROM 관리);
```

① 2, 3

② 1

③ 한 건도 반환되지 않는다.

④ 1, 2, 3

정답 ③

핵심 해설

NOT IN은 내부적으로 '사원번호 <> 1 AND 사원번호 <> NULL' 형태로 전개된다. NULL과의 비교(<>)는 참도 거짓도 아닌 UNKNOWN을 반환하고, AND에 UNKNOWN이 섞이면 전체 조건이 결코 TRUE가 될 수 없다. 그 결과 모든 행이 탈락하여 한 건도 반환되지 않으므로 정답은 ③. 오답: ①(2,3)은 NULL을 무시했을 때의 잘못된 기대값, ②·④도 성립할 수 없다.

> **보충 개념** 이것이 유명한 'NOT IN + NULL' 함정이다. 서브쿼리에 NULL이 포함될 수 있으면 NOT EXISTS를 쓰거나 서브쿼리에서 WHERE 관리자번호 IS NOT NULL 로 NULL을 제거해야 한다. NOT EXISTS는 NULL에 안전하다.

49 2과목

DCL(GRANT·REVOKE)과 권한에 대한 설명으로 옳지 않은 것은?

① GRANT SELECT ON 사원 TO 사용자A; 는 사용자A에게 사원 테이블 조회 권한을 부여한다.

② `WITH GRANT OPTION`을 붙여 권한을 부여하면, 권한을 받은 사용자가 다시 다른 사용자에게 그 권한을 부여할 수 있다.

③ REVOKE는 이전에 부여한 권한을 회수하는 명령이다.

④ ROLE에 부여한 권한은 한 번 부여되면 REVOKE로 회수할 수 없다.

정답 ④

핵심 해설

정답은 옳지 않은 진술인 ④. ROLE은 여러 권한을 묶은 집합으로, ROLE에 부여한 권한도 REVOKE로 회수할 수 있고 ROLE 자체도 사용자로부터 REVOKE할 수 있다. ①(참): GRANT 객체권한 ON 객체 TO 사용자 는 표준 객체 권한 부여 구문이다. ②(참): WITH GRANT OPTION은 권한의 재부여(전파)를 허용하며, 이 상태에서 부여자의 권한이 회수되면 연쇄적으로 회수(cascade)될 수 있다. ③(참): REVOKE는 GRANT의 반대로 권한을 회수한다.

> **보충 개념** 권한은 시스템 권한(예: CREATE SESSION)과 객체 권한(예: SELECT ON 테이블)으로 나뉘며, ROLE을 쓰면 다수 사용자의 권한 관리를 단순화할 수 있다.

50 2과목

다음 SQL을 실행한 결과로 옳은 것은? (사원 테이블에는 총 10건의 행이 있다.)

```
SELECT *
FROM 사원
WHERE 1 = 0;
```

① 사원 테이블의 전체 10건이 조회된다.

② 문법 오류가 발생하여 실행되지 않는다.

③ 1 = 0 은 허용되지 않으므로 1 = 2 로 바꿔야 실행된다.

④ 조건이 항상 거짓이므로 한 건도 반환되지 않는다(공집합).

정답 ④

핵심 해설

WHERE 1 = 0 은 상수끼리의 비교로 어떤 행에 대해서도 항상 거짓(FALSE)이다. 따라서 모든 행이 필터링되어 결과는 0건, 즉 공집합이 되므로 정답은 ④.
①은 항상 참(1=1)일 때의 결과, ②·③은 문법 오해다 — 1=0 이나 1=2 모두 문법적으로 완전히 유효한 표현식이며 실행된다.

> **보충 개념** WHERE 1=0 은 '데이터는 필요 없고 컬럼 구조만 복사'하려는 CTAS(CREATE TABLE ... AS SELECT * FROM t WHERE 1=0) 등에서 빈 껍데기 테이블을 만드는 관용적 기법으로 활용된다.